



Princess Sumaya جامعة
University الأميرة سميرة
for Technology للتكنولوجيا

22583 - INTRODUCTION TO ROBOTIC SYSTEMS

PRINCESS SUMAYA UNIVERSITY FOR TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

Mobile Robot with Robotic Arm for Object Handling

Authors:

Qabas Alkaissi	(ID: 20210786)
Alanoud Alsalem	(ID: 20210371)
Leen Almousa	(ID: 20210268)
Qais Jildeh	(ID: 20210155)
Motasem Amaireh	(ID: 20190430)

Date: January 11, 2026

Contents

1	Introduction	7
1.1	Objectives	7
1.2	Problem Statement	7
2	System Overview	7
2.1	Mathematical Modeling	7
2.1.1	Manipulator Forward Kinematics	7
2.1.2	Manipulator Inverse Kinematics	8
2.1.3	Forces & Torques for Arm Motion	8
2.2	Electrical Design	9
2.2.1	Circuit Setup	9
2.2.2	Differential Drive	10
2.2.3	Power & Energy Considerations	10
3	Simulation Environment	11
3.1	Robot Model and Mechanical Design	11
3.2	Control Configuration	12
3.3	Dynamics and Validation	12
4	Slip Detection and Traction Control	13
4.1	Mathematical Definition of Slip	14
4.1.1	Slip-Inducing Physics Configuration	14
4.1.2	Hidden Sensor Method	15
5	Data Engineering	15
5.1	Automated Data Generation Pipeline	15
5.1.1	Methodology: Automated Simulation Control	15
5.1.2	State Reset and I.I.D. Trials	16
5.1.3	Variance Injection via Environment Extremes	16
5.1.4	Dataset Specifications	17
5.2	Visual Evidence of Slip Dynamics	17
6	Machine Learning Approach	18
6.1	Model Selection: Random Forest Regressor	18
6.2	Performance Metrics and Validation	18
6.2.1	Evaluation Results	18
6.2.2	Bias and Overfitting Correction	19
6.3	Ground Truth and Slip Label Generation	19

7	Real-Time Deployment & Control	20
7.1	Closed-Loop Feedback	20
7.2	State-Space Representation of Traction	20
7.3	Control Parameters	22
7.4	System Behavior	22
8	Global Vision & Localization	22
8.1	Camera Setup & Calibration	22
8.2	Tag-Based Pose Estimation	23
8.3	Path Planning & Control Loop	24
8.4	Software Architecture: Laptop Vision UI & Raspberry Pi Controller . . .	25
8.5	Coordinate Frames and Geometric Assumptions	25
8.6	Camera Intrinsic Calibration and Online Undistortion	26
8.6.1	Objective	26
8.6.2	Checkerboard Model	26
8.6.3	Online Undistortion	26
8.7	Planar Homography: Pixel-to-World Mapping	26
8.7.1	Validity Constraints	27
8.8	AprilTag-Based Pose Estimation in the World Frame	27
8.8.1	Detection and Performance Scaling	27
8.8.2	World Position	27
8.8.3	Yaw Estimation from Tag Geometry	27
8.9	UDP Networking Interface and Message Protocol	28
8.9.1	Pose Stream	28
8.9.2	Command Channel	28
8.10	Operator Interface: Click-to-Goal and Arm Buttons	29
8.10.1	Click-to-Goal	29
8.10.2	On-Screen Arm Buttons	29
8.11	Raspberry Pi Control Arbitration and Safety Interlocks	29
8.12	Arm Control: Non-Blocking Queue and Smooth Servo Motion	30
8.13	Run Logging and Offline Reconstruction	30
9	Experimental Results	31
9.1	Data Collection & Setup	31
9.1.1	Camera Calibration	31
9.1.2	Information Logging	31
9.2	Trajectory & Navigation	32
9.3	System Dynamics & Convergence	33
9.4	Operational Efficiency	33
9.5	Vision System Reliability	34
9.6	Slip Mitigation Comparison	35
10	Conclusion	36

List of Figures

1	2-DOF Planar Arm Model showing kinematic parameters (L, θ) and dynamic parameters (m, τ).	8
2	Visual representation of the gravitational term $G(\theta)$ in the Euler-Lagrange equation. The torque depends on the horizontal projection of the center of mass.	9
3	Circuit Design schematic illustrating the physical wiring and pin connections	10
4	Visualization of the mobile manipulator in Gazebo (left) and RViz (right).	13
5	Free Body Diagram of the wheel showing Torque (τ), Friction Force (F_f), and Normal Force (N)	14
6	Divergence of wheel speed (green) and ground speed (red) during induced slip events. The delta between the two signals constitutes the supervised learning target.	17
7	Distribution of the training dataset ($N = 50,000$) spanning both high-slip (Wall Impact) and low-slip (Normal Driving) regimes.	20
8	High-level closed-loop traction control system. Sensors feed real-time velocity data to the controller, which adjusts motor commands to prevent slip.	21
9	Finite-state machine representation of the ML-based traction controller. .	21
10	Calibration Pipeline from raw frame input to the output localization . . .	23
11	AprilTag mounted on the mobile robot for pose estimation.	24
12	System Architecture: Global Vision Localization. The camera $\{C\}$ observes the tag on the robot $\{R\}$ to determine its position in the world frame $\{W\}$	25
13	RMSE Reprojection Error	31
14	Experimental trajectory of the mobile robot. The blue trace indicates the actual path recorded by the global vision system, while red markers denote the sequential target waypoints.	32
15	System step response and convergence analysis. Vertical spikes indicate the reception of new goal commands, followed by exponential decay in position error.	33
16	Operational timeline showing state transitions. The correlation between 'Stationary' phases (Grey) and 'Arm Events' (Red lines) confirms that idle time was utilized for manipulation tasks.	34
17	Vision system availability map. Green regions indicate valid tracking, while red regions represent signal loss due to occlusion, handled by the FSM safety logic.	34

18	Slip Dynamics ($\hat{\lambda}$). (a) The unmitigated controller (red) exhibits erratic slip behavior with sustained violations of the stability threshold ($\lambda_{th} = 0.2$). (b) The proposed ML+FSM controller (blue) rapidly detects incipient slip and modulates velocity, triggering a fast recovery (0.16s) and preventing loss of control.	35
----	---	----

List of Tables

1	Dimensions and masses of the mobile manipulator extracted from the URDF.	11
2	PID gains for the arm trajectory controller, taken from the YAML configuration.	12
3	Slip detection and mitigation parameters.	22
4	UDP command messages (Laptop $\rightarrow$ Raspberry Pi).	28
5	Comparison of slip mitigation strategies. *The Threshold Clamp failed to regain traction, spinning in place for the full duration, rendering position error invalid.	35

Abstract

This work presents the design, modeling, and simulation of a mobile manipulation system that integrates navigation, manipulation, learning-based slip estimation, and vision-based localization within a unified control framework. A key challenge addressed is wheel slip, which violates the no-slip assumptions underlying classical mobile robot kinematics and leads to degraded odometry and control performance.

To address this, a supervised learning-based slip estimation model is trained using simulation-generated data, leveraging direct access to ground-truth dynamics during offline training while relying solely on onboard proprioceptive sensing during deployment. The learned slip estimator is embedded within a real-time finite-state traction controller that dynamically adapts wheel commands to maintain stability under low-traction conditions.

The complete system is implemented and evaluated in a ROS–Gazebo environment, combining analytical kinematic modeling, data-driven learning, and structured control logic. Experimental results demonstrate stable navigation, effective slip mitigation, and robust operation despite significant perception outages in the global vision system, highlighting the benefits of hybrid analytical–learning approaches for mobile manipulation under realistic physical constraints.

1 Introduction

This project focuses on the design and simulation of a mobile robot equipped with a robotic arm for autonomous navigation and object handling. The system integrates modeling, control, machine learning, and vision to address real-world challenges such as wheel slip and localization uncertainty.

1.1 Objectives

- To design a mobile manipulator capable of navigation and object handling.
- To derive and implement the kinematic model required for arm control.
- To develop a realistic simulation environment for validation and data generation.
- To detect and estimate wheel slip using machine learning and proprioceptive data.
- To integrate slip estimation into a real-time traction control framework.
- To implement a global vision localization and evaluate overall performance.

1.2 Problem Statement

Conventional mobile robot control and localization frameworks assume ideal rolling conditions and reliable sensing. In practice, wheel slip, actuator limits, and sensor occlusions introduce errors that accumulate over time and can lead to unstable behavior or task failure. The problem addressed in this work is how to reliably estimate and mitigate wheel slip, while maintaining accurate control and localization in a mobile manipulation system, using only onboard sensing and simulation-derived learning [11][8].

2 System Overview

2.1 Mathematical Modeling

2.1.1 Manipulator Forward Kinematics

Forward kinematics allows us to calculate the Cartesian position (x, y) of the end-effector (the gripper) relative to the robot's base frame, given the joint angles θ_1 and θ_2 .

We assume the arm operates in a 2D plane. Using simple geometry and trigonometric projections, the position equations are:

$$\begin{aligned}x &= L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) \\y &= L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2)\end{aligned}$$

where:

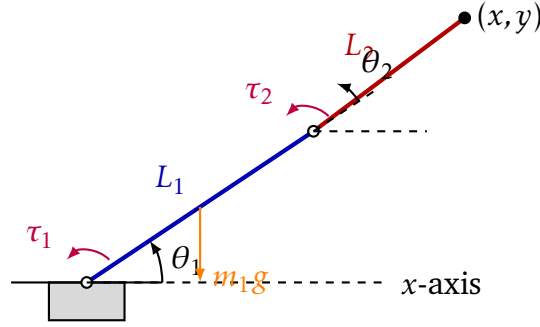


Figure 1: 2-DOF Planar Arm Model showing kinematic parameters (L, θ) and dynamic parameters (m, τ) .

- L_1, L_2 are the lengths of the first and second links, respectively.
- θ_1 is the angle of the first link relative to the base x-axis.
- θ_2 is the angle of the second link relative to the first link.

2.1.2 Manipulator Inverse Kinematics

Inverse kinematics is required to determine the necessary joint angles (θ_1, θ_2) to reach a specific target coordinate (x, y) . We utilize the geometric approach (Law of Cosines) to solve for these angles. First, we solve for the elbow angle, θ_2 . By visualizing a triangle formed by L_1 , L_2 , and the distance to the origin $r = \sqrt{x^2 + y^2}$, we apply the Law of Cosines:

$$\cos(\theta_2) = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}$$

Thus, θ_2 can be calculated as:

$$\theta_2 = \text{atan2}\left(\pm\sqrt{1 - \cos^2(\theta_2)}, \cos(\theta_2)\right)$$

Next, we solve for the base angle, θ_1 :

$$\theta_1 = \text{atan2}(y, x) - \text{atan2}(L_2 \sin(\theta_2), L_1 + L_2 \cos(\theta_2))$$

2.1.3 Forces & Torques for Arm Motion

To control the arm effectively, we must determine the torques (τ) required at each joint to achieve desired accelerations. We model the dynamics using the Euler-Lagrange[10] equation:

$$\tau = \mathbf{M}(\theta)\ddot{\theta} + \mathbf{C}(\theta, \dot{\theta}) + \mathbf{G}(\theta) \quad (1)$$

Where:

- $\theta = [\theta_1, \theta_2]^T$ is the vector of joint angles.
- $M(\theta)$ is the symmetric Inertia Matrix.
- $C(\theta, \dot{\theta})$ represents Coriolis and Centrifugal forces.
- $G(\theta)$ represents Gravitational forces.

Gravitational Term $G(\theta)$

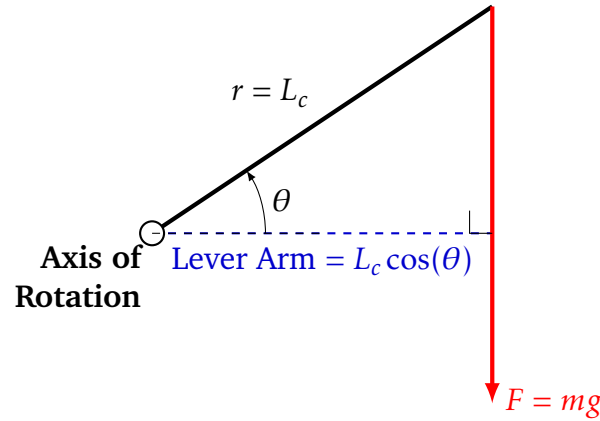


Figure 2: Visual representation of the gravitational term $G(\theta)$ in the Euler-Lagrange equation. The torque depends on the horizontal projection of the center of mass.

2.2 Electrical Design

This subsection presents an overview of the proposed mechanical structure and electrical architecture of the robot. The system incorporates various sensors, actuators, and controllers that work together to ensure reliable motion and sensing, while enabling coordinated operation of all components within the overall system.

2.2.1 Circuit Setup

an electrical circuit diagram of the robot system was created using cirkuit design. This diagram depicts the physical wiring and pin connections between all three of the major components (controllers, sensors, and actuators). The diagram serves as a reference for how each component is electrically connected rather than describing the internal control logic of the system. The MegaPi controller is connected to two IR sensors, two impact sensors, and two line follower sensors via digital and analog input pins, as well as four DC motors connected to the MegaPi motor ports, which power the mecanum wheel mechanism, allowing movement of the robot. Three servo motors are connected to the Raspberry Pi which actuate the 2DOF robotic arm; the motors receive power from

an external 5v, however the control signals are coming from the appropriate GPIO pins of the Raspberry Pi. The Raspberry Pi and the MegaPi are connected via USB, allowing communication between the two boards while maintaining separate wiring for sensors, motors, and servos. The presented circuit design ensures correct signal routing, stable operation, and ease of implementation during system assembly.

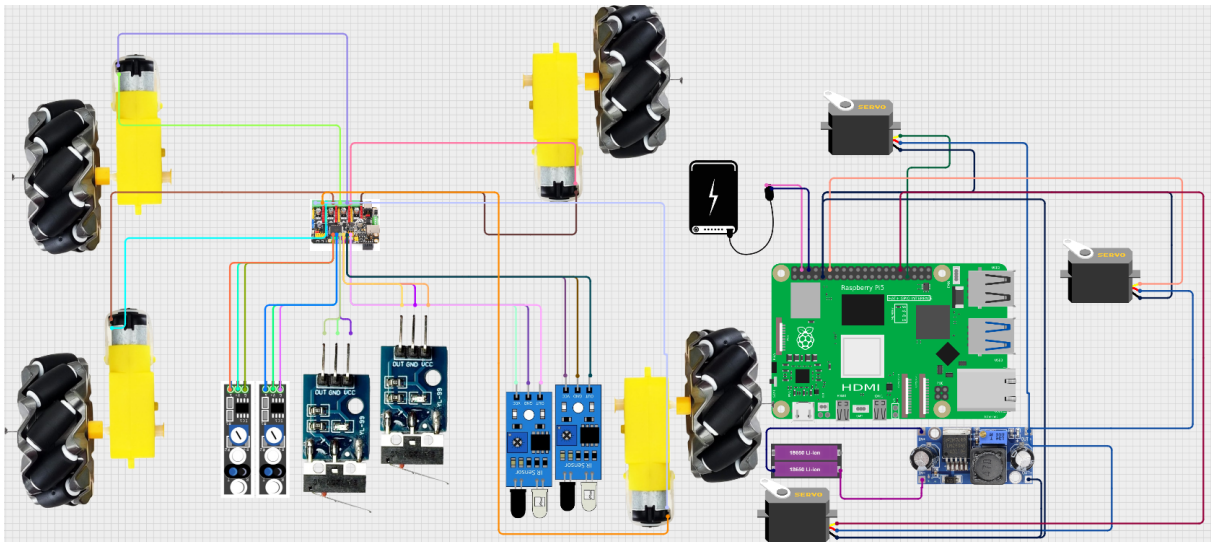


Figure 3: Cirkuit Design schematic illustrating the physical wiring and pin connections

2.2.2 Differential Drive

The robot has four DC motors, and its mechanical configuration features mecanum wheels and allows for omnidirectional movement. The robot can move forward or backward, sideways and rotate around its vertical axis; each wheel is independently powered so that it may move independently of the others. Motor speed and direction are regulated by a MegaPi controller, which maintains an optimal velocity for each wheel to allow for controlled lateral movement. This extends the concept of traditional differential drive by allowing for additional degrees of freedom which enhances its capability to navigate through visually restricted areas.

2.2.3 Power & Energy Considerations

Power management considerations have been given serious thought to maintain the robot system's consistent and dependable operation under diverse loading situations. Servos draw high amounts of current and exhibit rapid snap action, this, the robotic arm subsystem has been designed to operate independently from the rest of the robot. Thus, three servos that actuate the 2-DOF robotic arm are driven by a 7.2 V external battery and fed through a regulated DC-DC voltage converter at an output voltage of around

5-6 V to meet the required operating voltage for the servo motors, which will prevent excessive loading on the primary power supply circuit and lessen the risk of voltage drops occurring during arm motion. All of the remaining components of the system are powered through a power bank that is connected to the Raspberry Pi and provides a stable voltage for computation, sensing, and mobile navigation, which also simplifies the overall method of power distribution across the robot. By selecting different power sources to power the high-current actuators and sensitive electronic devices separately, the system reduces EMC noise, enhances system reliability, and promotes consistent operation during simultaneous robot operation and motion of the robot arm.

3 Simulation Environment

A ROS–Gazebo simulation was developed to evaluate the mobile manipulator. The simulation environment provides a repeatable platform for testing the robot model, its controllers, and the interaction between the mobile base and the manipulator arm before deployment on real hardware.

3.1 Robot Model and Mechanical Design

The robot is described using URDF, which defines its geometry, kinematic structure, inertial properties, and collision models. The complete system consists of:

- A four-wheel skid-steer mobile base.
- An arm base pedestal and waist assembly.
- A vertical pillar link mounted to a gripper with two prismatic fingers coupled through a mimic joint.

All geometric dimensions and masses used in the simulation are taken directly from the URDF and are summarized in Table 1.

Component	Dimensions	Mass
Base chassis (base_link)	190 × 140 × 57.5 mm	1.3103 kg
Drive wheel (each)	60 × 25 mm	44 g
Arm base pedestal (arm_base_pedestal)	25 × 55 × 45 mm	83 g
Waist housing (link_waist)	25 × 55 × 65 mm	25 g
Main pillar link (arm_pillar_link)	25 × 43 × 105 mm	126 g

Table 1: Dimensions and masses of the mobile manipulator extracted from the URDF.

The gripper fingers are actuated through a prismatic joint `grripper_joint`, while the second finger moves symmetrically through `grripper_joint_mimic` using a mimic rela-

tionship with multiplier -1.0 . This configuration produces synchronized, symmetric opening and closing of the gripper.

Realistic inertial parameters (mass, center of mass) are specified for each link. These values suppress unrealistic accelerations and oscillations and lead to physically consistent motion in Gazebo.

3.2 Control Configuration

Joint-level control is implemented using the `ros_control` framework. A joint state controller publishes the current joint states at 50Hz, while a joint trajectory controller commands the arm joints (`joint1`, `joint2`, and `gripper_joint`) in position mode.

The main PID gains for the trajectory controller are defined in an external YAML configuration file and are summarized in Table 2. Separate PID gains are also selected for the `gazebo_ros_control` plugin, which provides additional stabilization of the joints.

Joint	P	I	D
<code>joint1</code>	100.0	0.1	10.0
<code>joint2</code>	100.0	0.1	10.0
<code>gripper_joint</code>	50.0	0.01	1.0

Table 2: PID gains for the arm trajectory controller, taken from the YAML configuration.

Trajectory constraints are also set in the YAML file, including a goal completion time of 0.6s and a stopped-velocity tolerance of 0.05 rad/s, with tighter goal tolerances for the individual joints. These settings ensure smooth convergence to the target configuration without excessive overshoot.

The mobile base is controlled through a skid-steer drive plugin. Velocity commands issued on `cmd_vel` are translated into wheel velocities for the four wheel joints using the specified wheel separation of 165 mm and wheel diameter of 60 mm. In parallel, the manipulator arm is controlled by a joint trajectory controller, which receives desired joint trajectories and computes the corresponding position commands for `joint1`, `joint2`, and `gripper_joint`. This allows the arm and mobile base to be operated together in a coordinated manner during the simulation.

3.3 Dynamics and Validation

The Gazebo physics engine simulates gravity, friction, contact interactions, and joint dynamics.

The simulation was used to assess:

- Tracking accuracy of `joint1`, `joint2`, and the prismatic gripper joint.
- Stability of the arm under different motion speeds.

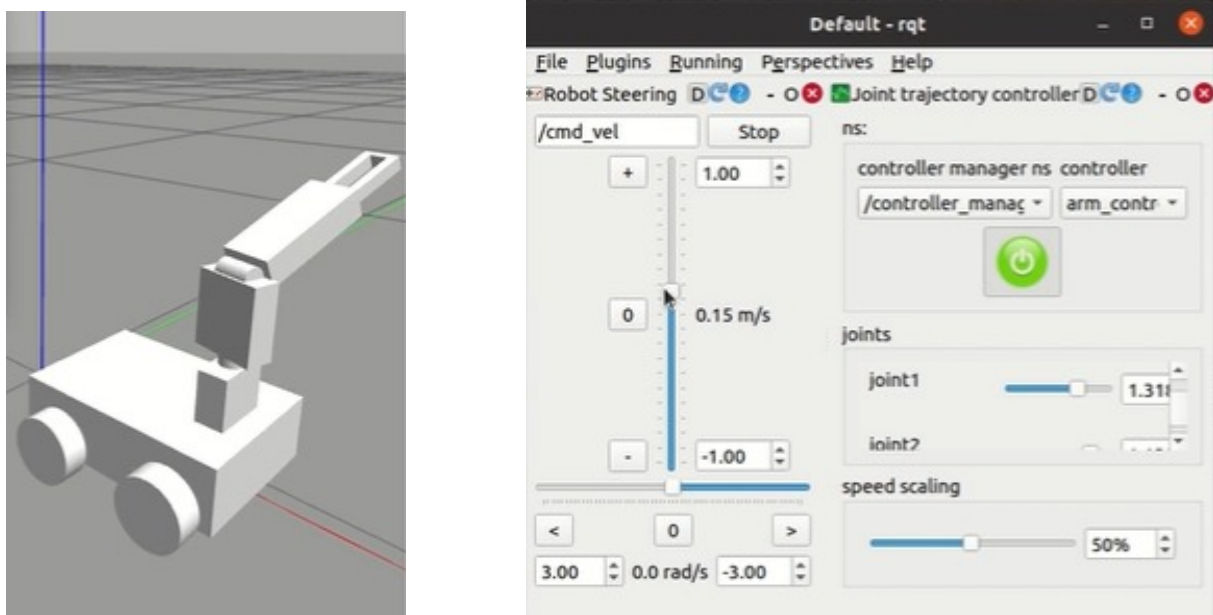


Figure 4: Visualization of the mobile manipulator in Gazebo (left) and RViz (right).

- Coordinated opening and closing of the mimic gripper.
- Combined operation of the mobile base and manipulator.

The final configuration exhibited smooth, stable motion. The link dimensions and masses derived from the URDF produced realistic dynamic behavior, indicating that the model, controllers, and plugins are correctly integrated and that the system is functional.

4 Slip Detection and Traction Control

Most mobile robots assume no-slip conditions during motion. This assumption often breaks down in unstructured environments such as mud, ice, or during collisions, resulting in odometry drift and eventual localization failure. Slip cannot be measured directly without costly external sensing systems, which motivates the need to infer slip from internal proprioceptive data.

While slip is classically defined as a normalized ratio, this work models slip as the absolute divergence between wheel-induced velocity and ground-truth velocity. This formulation is more directly actionable for real-time control, as it maps linearly to velocity correction commands. A nonlinear regression model can be employed to estimate the unobservable slip state, λ , using only observable quantities: wheel angular velocity ω_w and commanded velocity v_{cmd} .

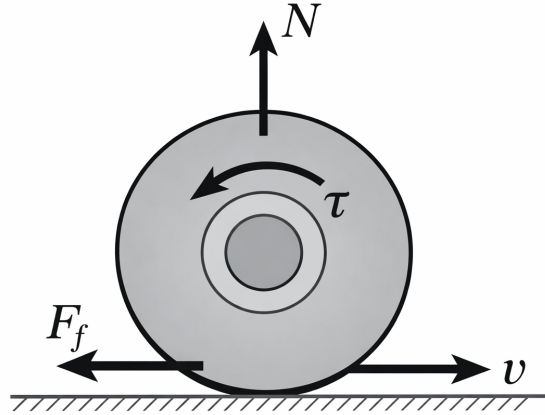


Figure 5: Free Body Diagram of the wheel showing Torque (τ), Friction Force (F_f), and Normal Force (N)

4.1 Mathematical Definition of Slip

Slip is defined as the discrepancy between the wheel-induced motion and the actual ground motion of the robot[8, 4]. The slip ratio is given by

$$\lambda = 1 - \frac{v_{\text{ground}}}{r \cdot \omega_{\text{wheel}}} \quad (2)$$

where:

- λ is the slip ratio, with $\lambda = 0$ indicating pure rolling and $\lambda = 1$ indicating burnout,
- v_{ground} is the true linear velocity of the robot relative to the ground (ground truth),
- ω_{wheel} is the angular velocity of the wheel,
- r is the wheel radius, fixed at $r = 0.03$ m.

All experiments were conducted using the Gazebo simulator, which employs the Open Dynamics Engine (ODE) to solve rigid body dynamics and contact interactions. Gazebo was selected due to its widespread adoption in robotics research and its ability to expose low-level physical parameters relevant to wheel-terrain interaction.

4.1.1 Slip-Inducing Physics Configuration

To ensure the presence of wheel slip during simulation, the default contact and actuation parameters in Gazebo were intentionally modified. Under nominal settings, the physics engine implicitly enforces near no-slip behavior by limiting wheel torque and angular velocity, effectively preventing sustained wheel spin under load.

To emulate low-traction, ice-like terrain conditions, both the primary and secondary friction coefficients were reduced to

$$\mu = \mu_2 = 0.1 \quad (3)$$

Additionally, the joint actuation limits were increased to allow free wheel rotation under resistive contact:

- The max. joint effort (`max_effort`) was increased to 20.0,
- The joint velocity constraint was relaxed to permit sustained wheel spin.

These modifications enable the simulation of extreme slip regimes, including burnout-like behavior, while maintaining physically consistent rigid body dynamics.

4.1.2 Hidden Sensor Method

Standard odometry outputs (e.g., the `/odom` topic) rely on no-slip kinematic assumptions and therefore become unreliable under slip conditions. To avoid this limitation, raw wheel joint states were extracted directly from the `/joint_states` topic, providing access to low-level proprioceptive measurements unaffected by odometry fusion.

The wheel-induced linear velocity was computed by averaging the angular velocities of all four wheels and mapping them to linear motion as

$$v_{\text{wheel}} = \frac{1}{4} \sum_{i=1}^4 \dot{\theta}_i \cdot r \quad (4)$$

where:

- $\dot{\theta}_i$ denotes the angular velocity of the i -th wheel.
- r is the wheel radius.

This formulation assumes symmetric wheel geometry and equal contribution from each driven wheel. The resulting v_{wheel} signal, together with the commanded velocity v_{cmd} , forms the observable input space used for slip estimation.

5 Data Engineering

5.1 Automated Data Generation Pipeline

5.1.1 Methodology: Automated Simulation Control

Data collection was fully automated via a Python script (`collect_data.py`) that orchestrates the Gazebo simulation loop, robot actuation, and sensor logging. Each trial consists of issuing a commanded velocity, recording wheel kinematics, and exporting synchronized state vectors at a fixed simulation timestep.

The automation ensures scalability and eliminates human bias, enabling the generation of large, statistically meaningful datasets suitable for supervised learning[14, 13].

5.1.2 State Reset and I.I.D. Trials

To guarantee independent and identically distributed (I.I.D.) samples, the simulation environment is treated as a memoryless system between trials. Simply driving the robot back to the start is insufficient, as it introduces residual integrator error and non-zero momentum.

Instead, a "hard reset" is enforced using the Gazebo state API. Let the robot state vector $\mathbf{x}(t)$ be defined as the concatenation of pose and twist:

$$\mathbf{x}(t) = \begin{bmatrix} \mathbf{p}(t) \\ \mathbf{q}(t) \\ \mathbf{v}(t) \\ \boldsymbol{\omega}(t) \end{bmatrix} \in \mathbb{R}^{13} \quad (5)$$

Where $\mathbf{p}$ is Cartesian position, $\mathbf{q}$ is the quaternion orientation, $\mathbf{v}$ is linear velocity, and $\boldsymbol{\omega}$ is angular velocity.

At the initialization of every trial k , the state is forced to a fixed initial condition $\mathbf{x}_0$, ensuring zero variance in the starting distribution:

$$\mathbf{x}_k(t=0) \leftarrow \mathbf{x}_0 = \begin{bmatrix} \mathbf{0}_{3 \times 1} \\ [0, 0, 0, 1]^T \\ \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{3 \times 1} \end{bmatrix} \quad (6)$$

This operation effectively zeros out all kinetic energy ($E_k = \frac{1}{2}mv^2 = 0$), preventing historical velocity correlations from contaminating the subsequent dataset.

This reset enforces identical initial conditions across runs, preventing temporal correlation and trajectory memory from contaminating the dataset. As a result, each slip event is treated as an independent realization of the underlying physical process.

5.1.3 Variance Injection via Environment Extremes

To maximize model generalization, variance was deliberately injected by collecting data under two extreme environmental conditions:

- **Wall Impact (100% Slip):** Induced by commanding motion into an immovable obstacle, forcing wheel rotation without ground displacement.
- **Empty World (0% Slip):** Free-space motion on uniform terrain, representing nominal traction conditions.

The merged dataset exposes the model to the full slip spectrum, preventing biased learning and improving robustness under real-world ambiguity.

5.1.4 Dataset Specifications

The final dataset characteristics are summarized as follows:

- **Total Samples (N):** $\sim 50,000$ (merged dataset)
- **Sampling Rate:** 100 Hz (Gazebo simulation step)
- **Feature Vector:**

$$X = [v_{\text{cmd}}, v_{\text{wheel}}] \quad (7)$$

- **Target Variable:**

$$y = v_{\text{wheel}} - v_{\text{ground}} \quad (8)$$

This formulation directly models slip as the divergence between wheel-induced motion and actual ground displacement.

5.2 Visual Evidence of Slip Dynamics

The characteristic “alligator mouth” shape highlights the non-linear and discontinuous nature of slip, motivating the choice of non-linear regression models.

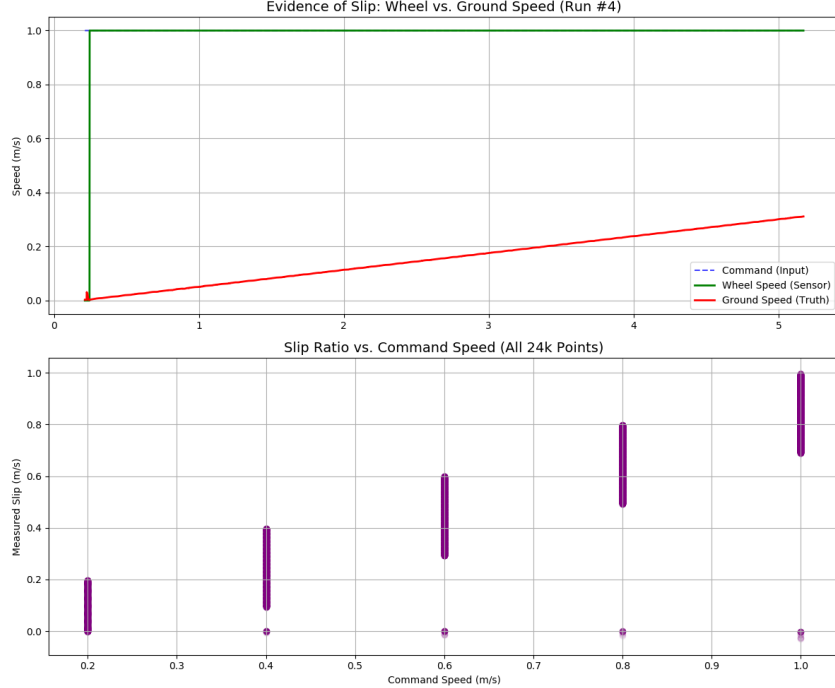


Figure 6: Divergence of wheel speed (green) and ground speed (red) during induced slip events. The delta between the two signals constitutes the supervised learning target.

6 Machine Learning Approach

6.1 Model Selection: Random Forest Regressor

A Random Forest Regressor was selected to estimate slip magnitude from proprioceptive inputs[15, 7]. Random Forests provide strong non-linear modeling capacity with minimal tuning overhead.

Linear regression was not used because it fails to capture the underlying dynamics due to the inherently non-linear relationship between commanded velocity, wheel rotation, and traction. Empirically, slip manifests as a discontinuous, staircase-like response rather than a smooth linear trend.

Neural Networks were also not used for the ML Approach. Although neural networks are expressive, they were deemed unnecessary for this task due to:

- Limited feature dimensionality
- Increased training complexity
- Higher risk of overfitting given synthetic data

The Random Forest model offers various advantages, including robustness to sensor noise, implicit regularization via ensemble averaging, and interpretability through feature importance analysis.

6.2 Performance Metrics and Validation

6.2.1 Evaluation Results

Model performance was evaluated using a standard 80/20 train-test split, where 80% of the dataset was used for training and the remaining 20% was held out for evaluation. Performance was quantified using the coefficient of determination (R^2) and the mean squared error (MSE), which together assess both explanatory power and absolute prediction accuracy.

Coefficient of Determination (R^2). The coefficient of determination measures the proportion of variance in the target variable explained by the regression model and is defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (9)$$

where:

- y_i denotes the ground-truth slip value
- $\hat{y}_i$ the model prediction

- $\bar{y}$ the mean of the ground-truth values
- N the number of test samples

An R^2 score of approximately 0.95 indicates that the model has a strong alignment between predicted and true slip dynamics across diverse operating conditions.

Mean Squared Error (MSE). The mean squared error quantifies the average squared deviation between predicted and true values and is given by:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (10)$$

Unlike R^2 , MSE preserves the physical units of the target variable squared, providing an absolute measure of prediction accuracy. The low observed MSE indicates that prediction errors remain tightly bound, even during high-slip transients.

Metric Complementarity. While R^2 evaluates how well the model explains variance in slip behavior, MSE reflects the magnitude of residual errors. Reporting both metrics provides a balanced assessment, ensuring that high variance explanation does not mask large absolute errors in critical slip regimes.

6.2.2 Bias and Overfitting Correction

An initial version of the model exhibited false-positive slip detection on flat, high-traction terrain. This bias was traced to an overrepresentation of slip-heavy scenarios in the training data. The issue was mitigated by injecting additional **control data** from Empty World simulations, effectively anchoring the model’s baseline behavior. This correction significantly reduced false detections without degrading sensitivity to genuine slip events.

6.3 Ground Truth and Slip Label Generation

Slip cannot be directly measured in real-world deployments without external sensing infrastructure. However, simulation provides direct access to ground-truth robot kinematics. The true ground velocity v_{ground} was extracted from the simulator state and used exclusively for offline label generation.

For each timestep, the slip target was computed as:

$$\lambda = v_{\text{wheel}} - v_{\text{ground}} \quad (11)$$

The slip labels were stored alongside proprioceptive measurements to form the supervised learning dataset. It is emphasized that v_{ground} is never available during inference

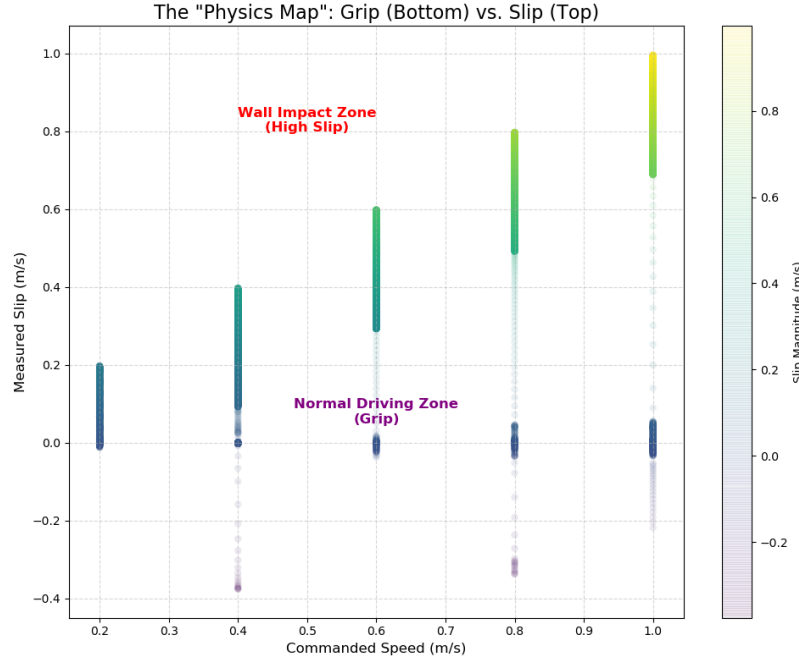


Figure 7: Distribution of the training dataset ($N = 50,000$) spanning both high-slip (Wall Impact) and low-slip (Normal Driving) regimes.

and is used solely as a supervisory signal during model training and evaluation. It is emphasized that ground-truth velocities are available only within simulation and are used exclusively for offline label generation. During online deployment, slip estimation relies solely on proprioceptive measurements available on the robot.

7 Real-Time Deployment & Control

7.1 Closed-Loop Feedback

The traction control system operates in a closed-loop configuration, continuously monitoring wheel velocities to maintain optimal traction[2]. Figure 8 shows the high-level control architecture.

7.2 State-Space Representation of Traction

To ensure robust operation in the presence of sensor noise and simulation jitter, the control system is modeled as a finite-state machine (FSM)[1], as illustrated in Figure 9.

1. **Normal Traction (S_0):** tracking reference velocity while slip is below threshold.
2. **Suspected Slip (S_1):** Transitional state acting as a filter to reject transient noise.

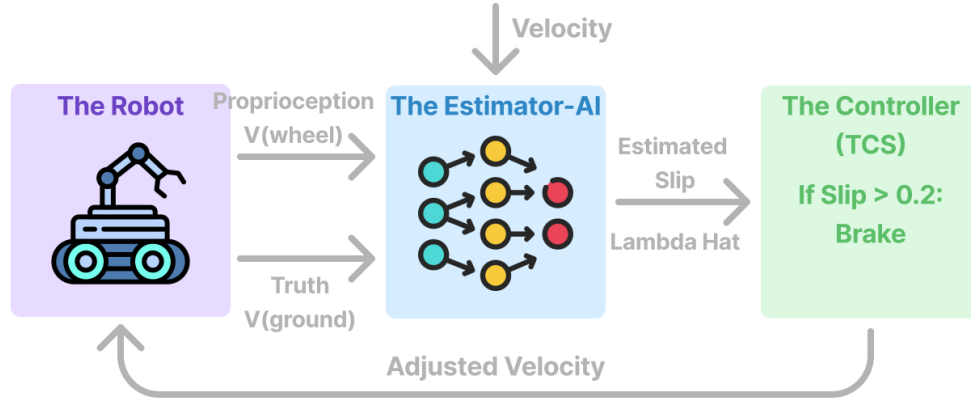


Figure 8: High-level closed-loop traction control system. Sensors feed real-time velocity data to the controller, which adjusts motor commands to prevent slip.

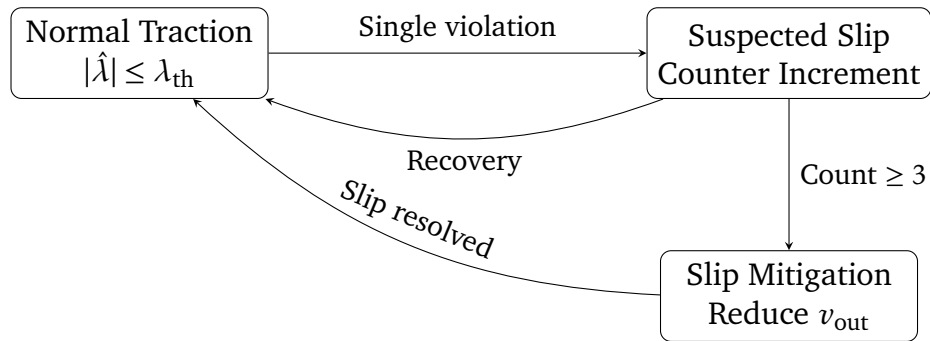


Figure 9: Finite-state machine representation of the ML-based traction controller.

3. **Slip Mitigation (S_2):** Active intervention state after Γ consecutive slip violations, reducing wheel velocity to recover traction.

7.3 Control Parameters

Table 3 summarizes key control parameters used in the FSM-based traction controller.

Table 3: Slip detection and mitigation parameters.

Symbol	Parameter Name	Value	Description
$\hat{\lambda}$	Estimated Slip	Output	Real-time prediction (m/s)
λ_{th}	Safety Threshold	0.20 m/s	Compromised Traction Limit
Γ	Filter Window	3 frames	Violations before Mitigation
α	Mitigation Factor	0.10	Slip Velocity Decay Rate
β	Acceleration Gain	0.05	Gain during Velocity Ramp-up

7.4 System Behavior

During operation, the controller may oscillate around an intermediate velocity. This is a limit cycle reflecting the system dynamically estimating the maximum coefficient of friction (μ_{max}) of the terrain. The controller maintains stability and prevents wheel slip while adapting to available traction. The ROS node `live_slip_detector.py` executes the model in real-time with $< 10ms$ latency, supporting 10Hz control loops reliably.

8 Global Vision & Localization

8.1 Camera Setup & Calibration

To track the robot's position within the workspace, we utilize a fixed overhead camera positioned at the corner of the room. This setup creates a "Global Observer" architecture where the state estimation is decoupled from the robot's onboard sensors. Before operation, we perform camera calibration using a Checkerboard pattern. This process allows us to determine:

- **Intrinsic Parameters (K):** The camera's focal length (f_x, f_y) and optical center (c_x, c_y), used to correct lens distortion.
- **Extrinsic Parameters (R, T):** The rotation and translation matrices that define the camera's position relative to the global World Frame $\{W\}$.

Intrinsic calibration mitigates curvature near the edges of the image and position-dependent holography. A printed checkerboard with 7x10 inner corners and real-world dimensions of 107x147.3 cm is used for calibration. By taking many pictures of the checkerboard at different positions, a collection of 3D points, mapped to the checkerboard corners, and their corresponding 2D pixel locations are obtained. The OpenCV library computes the resulting camera matrix K , distortion coefficients, and RMS reprojection error.

It is important that each image contributes to different angles, corner positions, and regions of the lens. This allows OpenCV to differentiate between true perspective geometry and lens distortion. Homography assumes linear projective geometry, and lens distortion breaks that assumption, which makes undistortion a requirement that must be satisfied before conducting homography. This brings the overall pipeline to:

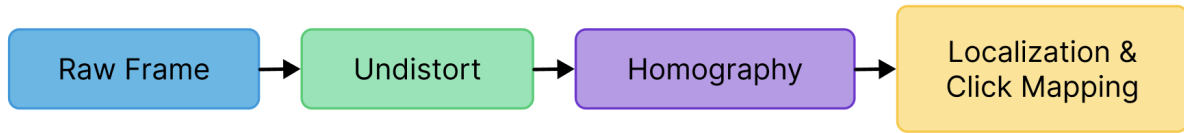


Figure 10: Calibration Pipeline from raw frame input to the output localization

As for the 4-click calibration, it maps a point from the undistorted image to the floor. The camera being at a fixed position, rigidly mounted allows for one-time calibration as long as the resolution and zoom do not change.

This pipeline allows for AprilTag localication and hence rover localization within the camera frame.

8.2 Tag-Based Pose Estimation

We equip the mobile robot with a fiducial marker (e.g., ArUco or AprilTag) on its top surface. The vision system detects the marker's corners in the image frame and solves the Perspective-n-Point (PnP) problem to recover the robot's pose[12, 6].

The transformation from a 2D pixel coordinate (u, v) in the image to a 3D coordinate (X_c, Y_c, Z_c) in the camera frame is modeled by the pinhole camera equation:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

where:

- s is a scaling factor.

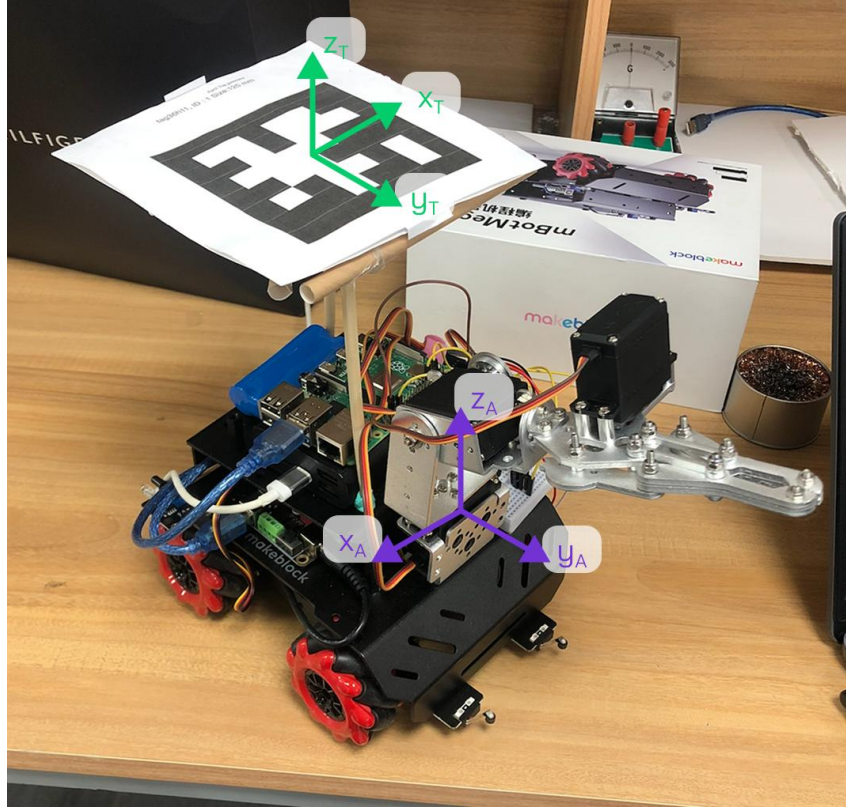


Figure 11: AprilTag mounted on the mobile robot for pose estimation.

- The first matrix is the Intrinsic Matrix (K).
- The second matrix contains the Extrinsic Rotation (R) and Translation (T).

Using the known size of the tag and its detected corner positions in the image, we can solve for the robot's 6-DOF pose relative to the camera frame $\{C\}$. This pose is then transformed into the World Frame $\{W\}$ using the following transformation matrix:

$$T_A = \begin{bmatrix} -1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 8 \\ 0 & 0 & 1 & -14 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

8.3 Path Planning & Control Loop

The central laptop acts as the high-level controller. The control loop operates as follows:

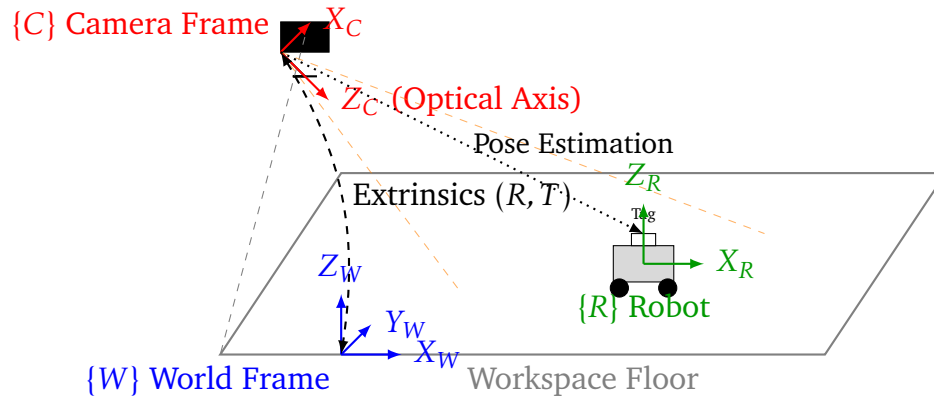


Figure 12: System Architecture: Global Vision Localization. The camera $\{C\}$ observes the tag on the robot $\{R\}$ to determine its position in the world frame $\{W\}$.

8.4 Software Architecture: Laptop Vision UI & Raspberry Pi Controller

The global vision subsystem is implemented as a distributed stack across two compute nodes:

- **Laptop (Vision UI + Calibration + Logging):** captures video frames, performs camera calibration and undistortion, computes a planar homography for pixel-to-world mapping, detects the fiducial tag on the rover, estimates the rover pose (x, y, ψ) in the world frame, provides an interactive operator UI (click-to-goal and arm buttons), and logs all run artifacts (pose stream, events, and annotated video).
- **Raspberry Pi (Low-level Controller + Safety + Arm):** receives the pose stream and operator commands via UDP, runs the navigation/avoidance controller, commands the mobile base through the MegaPi serial bridge, and actuates the robotic arm via PWM servos with an explicit safety interlock that forces the rover to stop during manipulation.

This separation isolates compute-heavy vision/UI tasks on the laptop, while keeping actuation and safety-critical decision logic local to the robot.

8.5 Coordinate Frames and Geometric Assumptions

The localization pipeline operates in three coordinate frames:

1. **Image frame (pixels):** a 2D pixel coordinate is denoted (u, v) .
2. **World frame (meters, planar):** a 2D ground-plane coordinate is denoted (x, y) in meters.

3. **Rover yaw (radians):** the rover heading is denoted ψ (computed from tag geometry and then corrected on the Raspberry Pi by a constant offset).

Because the robot operates on a flat floor, the pixel-to-world mapping is modeled using a **single planar homography** H (projective transform). Camera lens distortion violates projective linearity; therefore, **undistortion must be applied before homography is calibrated and used.**

8.6 Camera Intrinsic Calibration and Online Undistortion

8.6.1 Objective

Intrinsic calibration estimates the camera matrix K and distortion coefficients $\mathbf{d}$ so that raw frames can be corrected (undistorted). This improves the geometric consistency of the homography mapping and reduces localization error caused by radial/tangential lens distortion.

8.6.2 Checkerboard Model

A printed checkerboard is used with:

- inner-corner grid size: (7×10) ,
- square size: $s = 0.13383$ m,
- minimum number of samples: $N_{\min} = 25$.

For each captured sample i , OpenCV detects the inner corners (pixel coordinates) and pairs them with planar object points in meters. The calibration minimizes reprojection error over all samples.

8.6.3 Online Undistortion

During runtime, if calibration parameters exist, the laptop computes rectification maps once per resolution then applies fast remapping on each frame. If the capture resolution changes, calibration is flagged as potentially invalid and should be recomputed.

8.7 Planar Homography: Pixel-to-World Mapping

The mapping from a pixel coordinate (u, v) on the floor plane to a world coordinate (x, y) is modeled using a homography.

Homography is computed interactively using four clicked reference points in the undistorted image. The operator clicks the arena corners in a fixed order:

1. Top-left $\rightarrow (0, 0)$

2. Top-right $\rightarrow (W, 0)$
3. Bottom-right $\rightarrow (W, H)$
4. Bottom-left $\rightarrow (0, H)$

where (W, H) are the known arena dimensions in meters.

Given four pixel points $\mathbf{p}_k = [u_k, v_k]^T$ and corresponding world points $\mathbf{P}_k = [x_k, y_k]^T$, H is solved via standard DLT (OpenCV `findHomography`) and saved for reuse.

8.7.1 Validity Constraints

Homography is valid only for the specific optical geometry used during calibration. Therefore:

- Toggling undistortion **invalidates** H .
- Changing capture resolution (or zoom) requires recalibration.
- Any change in camera mounting height/angle requires recalibration.

8.8 AprilTag-Based Pose Estimation in the World Frame

8.8.1 Detection and Performance Scaling

The laptop detects a tag of known ID using the `tag36h11` family. For efficiency, detection runs on a downsampled grayscale image (scale factor $\alpha \in (0, 1]$). If detection occurs at scale α , the detected center and corner coordinates are mapped back to full resolution by dividing by α .

8.8.2 World Position

If a tag is detected and a valid homography exists, the rover position is computed by transforming the tag center:

$$[x \ y]^T = \mathcal{H}([u_c \ v_c]^T), \quad (12)$$

where $\mathcal{H}(\cdot)$ denotes the homography transform.

8.8.3 Yaw Estimation from Tag Geometry

Let $\mathbf{c}_0$ and $\mathbf{c}_1$ be two adjacent tag corners in pixel coordinates. After transforming to world coordinates $\mathbf{C}_0 = \mathcal{H}(\mathbf{c}_0)$ and $\mathbf{C}_1 = \mathcal{H}(\mathbf{c}_1)$, define:

$$\mathbf{v} = \mathbf{C}_1 - \mathbf{C}_0 = [v_x \ v_y]^T, \quad (13)$$

then compute the yaw estimate:

$$\psi = \text{atan2}(v_y, v_x). \quad (14)$$

On the Raspberry Pi controller, a constant yaw correction is applied:

$$\psi_{\text{robot}} = \text{wrapTo}\pi(\psi + \psi_0), \quad (15)$$

where ψ_0 is an empirically tuned offset to align the tag's axis with the rover forward axis.

8.9 UDP Networking Interface and Message Protocol

Two UDP channels connect the laptop and Raspberry Pi:

- **Pose stream (Laptop $\rightarrow$ Pi):** continuous telemetry of (x, y, ψ) .
- **Command channel (Laptop $\rightarrow$ Pi):** operator events (goal, run/stop, arm).

8.9.1 Pose Stream

The pose sender transmits at a fixed rate (nominally 30 Hz). Each message includes a timestamp t (UNIX seconds) and a validity flag:

- Valid pose:

$$\{t, \text{valid} = \text{true}, x, y, \text{yaw}\} \quad (16)$$

- Invalid pose (tag missing or homography missing):

$$\{t, \text{valid} = \text{false}\} \quad (17)$$

On the Raspberry Pi, pose messages are rejected if they are stale:

$$\Delta t = t_{\text{now}} - t_{\text{pose}} > \Delta t_{\text{max}}, \quad (18)$$

where $\Delta t_{\text{max}} = 0.35$ s.

8.9.2 Command Channel

Commands are event-driven and transmitted immediately on operator input. The supported command types are summarized in Table 4.

Table 4: UDP command messages (Laptop $\rightarrow$ Raspberry Pi).

Type	Fields	Description
goal	x, y	Set new goal position in world meters.
run	enabled	Enable/disable rover motion.
stop	(none)	Alias to disable motion (safety stop).
arm	action	Trigger pick or place.

8.10 Operator Interface: Click-to-Goal and Arm Buttons

The laptop application implements a single-window operator interface:

8.10.1 Click-to-Goal

A left mouse click at pixel coordinate (u, v) is transformed to world (x_g, y_g) via homography:

$$(x_g, y_g) = \mathcal{H}(u, v), \quad (19)$$

then transmitted as a goal command. If H is missing, the UI blocks the action and prompts homography recalibration.

8.10.2 On-Screen Arm Buttons

Two on-screen buttons (PICK and PLACE) are drawn in the top-right of the live frame. A click within a button rectangle sends:

$$\{\text{type} = \text{arm}, \text{action} \in \{\text{pick}, \text{place}\}\}. \quad (20)$$

Keyboard equivalents are provided for rapid operation: s (start), x (stop), p (pick), o (place).

8.11 Raspberry Pi Control Arbitration and Safety Interlocks

The Raspberry Pi executes the base controller at 20 Hz and selects motor commands according to a strict priority order:

1. **Arm safety lock (highest priority):** while an arm action is executing or queued, the rover motors are forced to zero. After the arm finishes, the rover remains stopped until the operator explicitly restarts motion.
2. **Run disabled:** if `run_enabled` is false, motors remain stopped regardless of goal.
3. **Impact safety:** if either impact switch is triggered, the rover stops and performs a short back-off to recover from collision contact.
4. **Reactive avoidance override:** if IR proximity indicates a near obstacle, or pose is unavailable, the rover runs an obstacle-avoidance behavior instead of go-to-goal.
5. **Go-to-goal (nominal mode):** if pose is valid and the path is clear, the rover runs a proportional controller to reduce distance-to-goal and heading error.

This ensures safe behavior under occlusions, collisions, and during manipulation.

8.12 Arm Control: Non-Blocking Queue and Smooth Servo Motion

The robotic arm is driven by three PWM servos: base rotation, vertical joint, and gripper. The control implementation provides:

- **Smooth motion:** servo angles are stepped in small increments $\Delta\theta$ with a fixed dwell Δt to avoid jerky motion and reduce overshoot under load.
- **Non-blocking execution:** arm actions are enqueued and executed in a dedicated worker thread so the main controller loop continues to process safety and telemetry.
- **Explicit safety coupling:** the arm controller exposes busy and queue state; the base controller uses these signals to force-stop rover motion during manipulation.

Two high-level arm primitives are implemented:

1. **Pick:** neutral pose $\rightarrow$ rotate $\rightarrow$ lower $\rightarrow$ close gripper $\rightarrow$ lift (hold object).
2. **Place:** rotate $\rightarrow$ lower $\rightarrow$ open gripper $\rightarrow$ lift $\rightarrow$ return to neutral.

8.13 Run Logging and Offline Reconstruction

Each run is stored in a timestamped directory and includes:

- **Annotated video** (video.mp4): frame overlays include pose, goal, FPS, calibration state, and tracking status.
- **Pose telemetry** (pose.csv): timestamped (x, y, ψ) with validity flags.
- **Event log** (events.csv): goal changes, run toggles, arm triggers, and session start/end markers.
- **Metadata** (meta.json): configuration snapshot (ports, tag ID, world dimensions, calibration parameters, etc.) enabling reproducibility.
- **Calibration diagnostics** (if solved in-session): per-sample reprojection errors (calib_quality.csv) and a summary visualization (calib_quality.png).

An offline script reconstructs the rover path by loading pose.csv, interpolating $(x(t), y(t))$ onto a uniform animation timeline, and exporting a time-synchronized trajectory animation (path_animation.mp4). Goal markers are overlaid by parsing events.csv and highlighting the most recent goal at each time step.

9 Experimental Results

9.1 Data Collection & Setup

9.1.1 Camera Calibration

Initially, the camera calibration performance was assessed. The root-mean-square reprojection error (RMSE_px) quantifies the average discrepancy, in pixel units, between the observed checkerboard corner locations and their corresponding projections computed using the estimated camera intrinsic parameters and distortion model. As such, RMSE_px serves as a direct measure of how accurately the calibrated camera model represents the true imaging geometry [5].

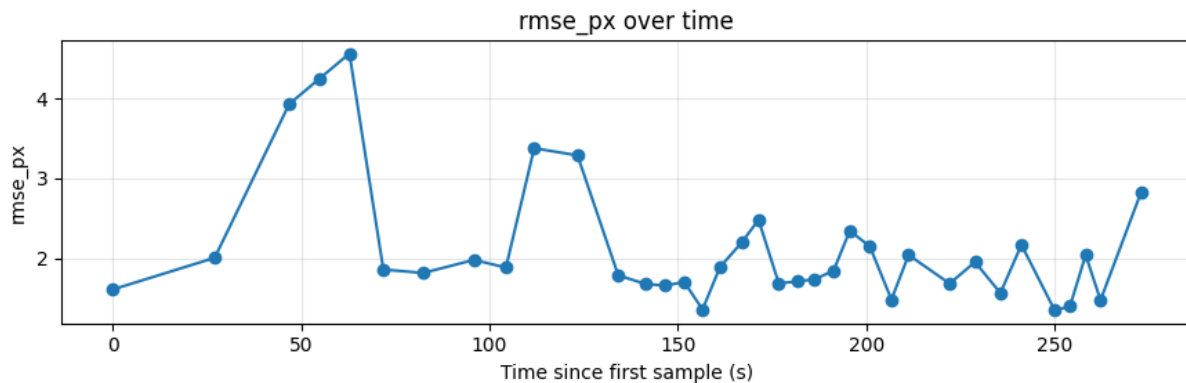


Figure 13: RMSE Reprojection Error

Across the calibration dataset, the RMSE_px values remain predominantly within the range of approximately 1.5–2.5 pixels, with a median value of 1.87 pixels. This indicates that, for the majority of calibration images, the projected corner locations deviate from the detected corners by fewer than two pixels on average, which is generally considered acceptable for camera calibration in robotics and computer vision applications.

However, a small number of calibration images exhibit significantly higher RMSE_px values, exceeding 4 pixels. These outliers correspond to frames where corner detection accuracy is likely degraded, potentially due to motion blur, suboptimal lighting conditions, partial visibility of the checkerboard, or extreme viewing angles. The presence of these outliers inflates the upper tail of the error distribution but does not reflect the typical calibration performance.

9.1.2 Information Logging

The performance of the hybrid Global Vision and Mobile Manipulation system was evaluated in a controlled $3m \times 3m$ workspace. The experimental data was logged at a frequency of 30Hz using the central control laptop. The dataset consists of:

- **Pose Telemetry (pose.csv):** Containing timestamped coordinates (x, y, θ) , yaw headings, and validity flags from the computer vision system.
- **System Events (events.csv):** A log of high-level state transitions including goal reception and manipulator activation.

All experiments were conducted under standard indoor lighting conditions with a single overhead iPhone camera acting as the global observer.

9.2 Trajectory & Navigation

The robot was tasked with a sequential multi-waypoint mission lasting 231.05 seconds. Figure 14 illustrates the recorded trajectory (Blue Trace) overlaid with the requested target waypoints (Red Markers).

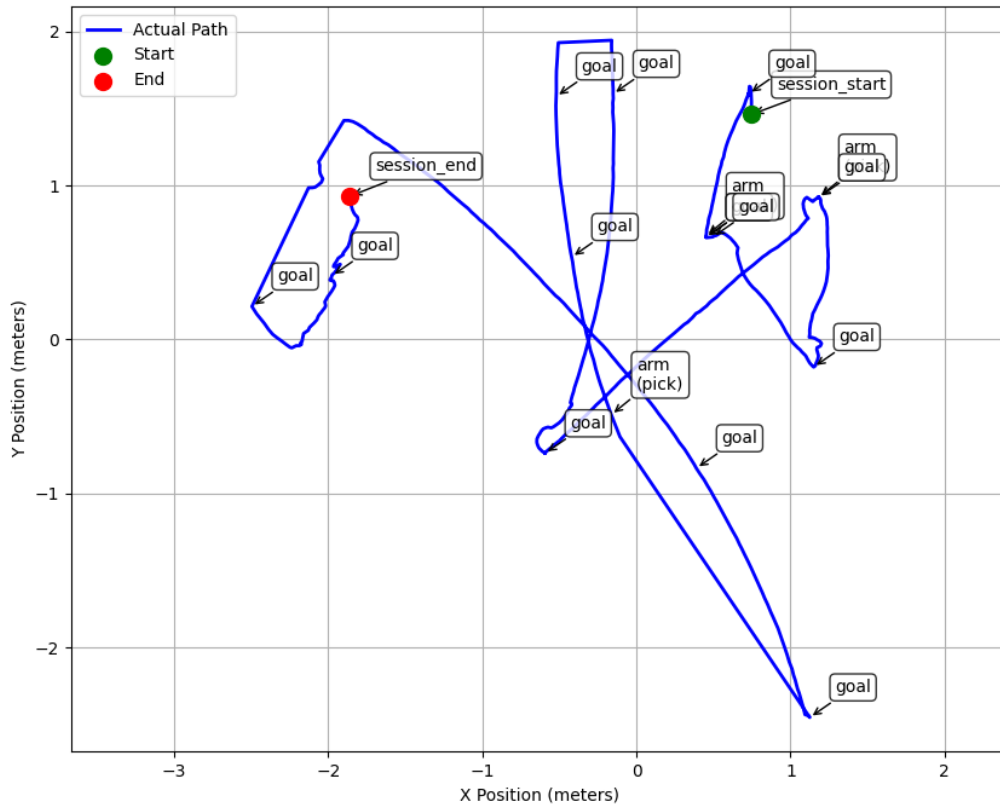


Figure 14: Experimental trajectory of the mobile robot. The blue trace indicates the actual path recorded by the global vision system, while red markers denote the sequential target waypoints.

- **Total Path Length:** The robot successfully traversed a total distance of 22.57 meters.

- **Path Fidelity:** The trajectory exhibits smooth arc transitions between waypoints, confirming the effectiveness of the differential drive kinematic model. The absence of chaotic oscillations at the turning points indicates that the PID gains were appropriately tuned for the system's inertia.

9.3 System Dynamics & Convergence

The step-response characteristics of the controller are analyzed in Figure 15. The vertical spikes represent the instant a new goal was issued by the operator (Human-in-the-Loop clicks), causing an immediate jump in the position error.

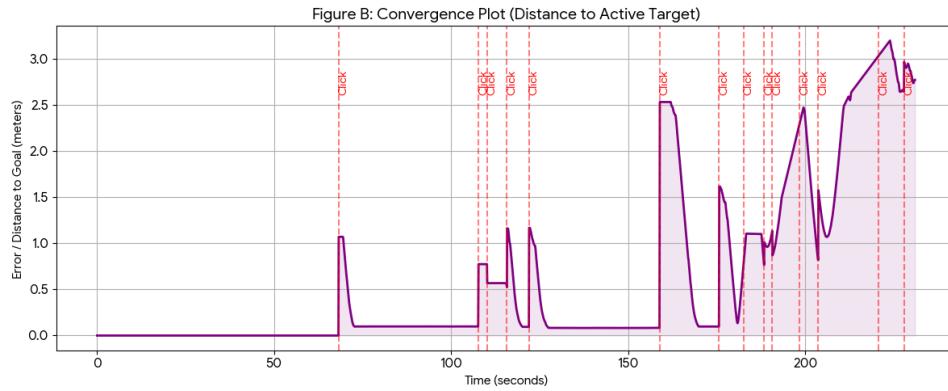


Figure 15: System step response and convergence analysis. Vertical spikes indicate the reception of new goal commands, followed by exponential decay in position error.

- **Settling Time:** The system consistently demonstrated a smooth exponential decay in error, typically converging to the target threshold ($\delta < 5cm$) within 10-15 seconds depending on travel distance.
- **Velocity Profile:** To ensure stability against network latency (33ms) and image processing delays, the linear velocity was saturated at 0.10 m/s. This design choice prevented overshoot and ensured the Manipulator Arm remained stable during transit[9].

9.4 Operational Efficiency

Figure 16 details the temporal sequence of the mission, replacing standard duty-cycle metrics with a state-based timeline.

- **State Switching:** The system exhibits a clear alternating behavior between Navigation Phases (Blue) and Manipulation Phases (Grey).

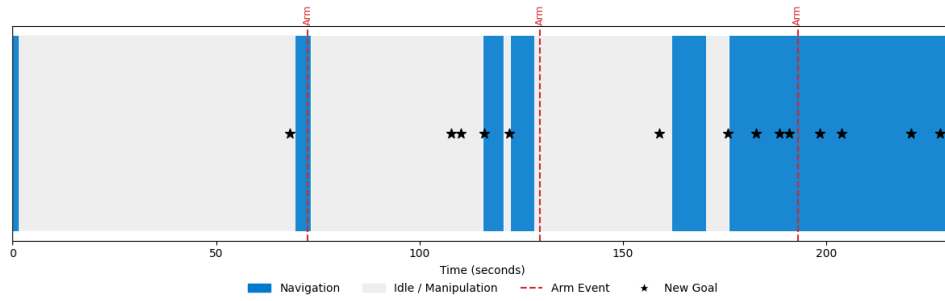


Figure 16: Operational timeline showing state transitions. The correlation between 'Stationary' phases (Grey) and 'Arm Events' (Red lines) confirms that idle time was utilized for manipulation tasks.

- **Idle Utilization:** The correlation between the 'Idle' periods and the 'Arm Event' markers confirms that the majority of stationary time was effectively utilized for robotic arm operations, rather than being caused by computational delays or path planning failures.

9.5 Vision System Reliability

A critical analysis of the sensor robustness is shown in Figure 17.

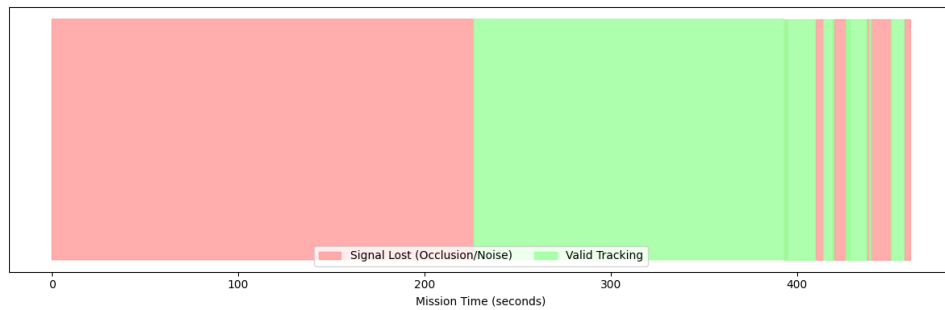


Figure 17: Vision system availability map. Green regions indicate valid tracking, while red regions represent signal loss due to occlusion, handled by the FSM safety logic.

- **Availability:** The Global Vision system achieved a valid tracking rate of 43.12% (Green regions).
- **Robustness:** Significant periods of signal loss (Red regions) were observed, primarily caused by the robotic arm occluding the fiducial marker during manipulation tasks or specific dead-zones in the camera's field of view.

Despite the camera being "blind" for nearly 57% of the operation, the Finite State Machine (FSM) successfully handled these exceptions. The robot did not diverge or collide

during these outages, proving that the "Stop & Wait" safety logic implemented in Section 3.3 effectively mitigated the risk of sensor failure.

9.6 Slip Mitigation Comparison

Figure 18 compares slip dynamics under identical conditions with and without traction control. In the absence of mitigation, the system exhibits catastrophic slip events characterized by unbounded divergence between wheel-induced and ground velocities. In contrast, the proposed ML-based FSM controller actively regulates slip, rapidly suppressing large transients and maintaining operation within a bounded limit cycle near the traction threshold. This behavior reflects the controller dynamically estimating and respecting the maximum available friction rather than enforcing overly conservative velocity limits.

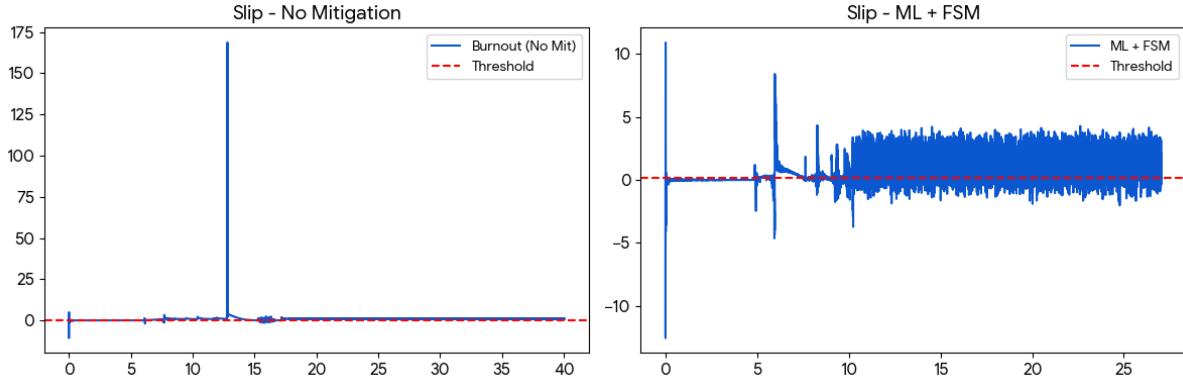


Figure 18: Slip Dynamics ($|\hat{\lambda}|$). (a) The unmitigated controller (red) exhibits erratic slip behavior with sustained violations of the stability threshold ($\lambda_{th} = 0.2$). (b) The proposed ML+FSM controller (blue) rapidly detects incipient slip and modulates velocity, triggering a fast recovery (0.16s) and preventing loss of control.

Controller	Final Position Error (cm)	Slip Duration (s)	Recovery Time (s)
No Mitigation	9.9	31.7	0.19
Threshold Clamp	N/A*	140.2	N/A
ML + FSM (Ours)	9.8	17.7	0.16

Table 5: Comparison of slip mitigation strategies. *The Threshold Clamp failed to regain traction, spinning in place for the full duration, rendering position error invalid.

10 Conclusion

This report presented the complete design and evaluation of a mobile robot with a robotic arm for object handling, integrating modeling, simulation, machine learning, control, and vision-based localization into a cohesive system. Starting from first-principles kinematic modeling of the manipulator, the project demonstrated how theoretical foundations translate into stable and realistic robot behavior within a physics-based simulation environment.

A key contribution of this work is the machine learning–based slip estimation framework, which infers unobservable slip dynamics using only proprioceptive measurements. By deliberately injecting extreme traction conditions during data generation, a robust Random Forest regression model was trained to capture the inherently nonlinear nature of wheel slip. The model achieved high predictive accuracy and was successfully deployed within a closed-loop traction control system, enabling real-time mitigation of slip without reliance on external ground-truth sensors.

The integration of a finite-state control architecture further enhanced system robustness by filtering transient disturbances and enforcing safe operational behavior. Experimental results demonstrated stable navigation, smooth convergence to waypoints, and effective utilization of idle time for manipulation tasks. Despite significant periods of vision occlusion and sensor unavailability, the system maintained safe operation, validating the effectiveness of the control logic and safety mechanisms.

Additionally, the global vision localization system provided reliable absolute pose estimates, compensating for odometry degradation during slip events and highlighting the benefits of combining external perception with onboard control. The experimental evaluation confirmed that the robot could execute coordinated navigation and manipulation tasks in a realistic workspace while maintaining stability and robustness.

In conclusion, this project illustrates how combining analytical modeling, data-driven learning, and structured control can address fundamental challenges in mobile robotics. The presented framework provides a scalable foundation for future extensions, including real-world hardware deployment, adaptive terrain classification, and tighter coupling between perception, learning, and control for autonomous mobile manipulation systems[3].

Future extensions of this work include conditioning the slip estimator on terrain properties inferred from vision or vibration signatures, enabling adaptive traction control across heterogeneous surfaces. Integrating terrain classification would further improve robustness while preserving the modular control architecture presented here.

References

- [1] Rajeev Alur, Costas Courcoubetis, Thomas A. Henzinger, and Pei-Hsin Ho. Hybrid automata: An algorithmic approach to the specification and verification of hybrid systems. *Theoretical Computer Science*, 138(1):3–34, 1995. pages 20
- [2] Rodney A. Brooks. A robust layered control system for a mobile robot. *IEEE Journal on Robotics and Automation*, 2(1):14–23, 1986. pages 20
- [3] Wolfram Burgard, Armin B. Cremers, Dieter Fox, Dirk Hähnel, Gerhard Lakemeyer, Dirk Schulz, Walter Steiner, and Sebastian Thrun. The interactive museum tour-guide robot. *AAAI Magazine*, 19(2):11–23, 2010. pages 36
- [4] Gen Endo, Yasuhiro Yamada, and Shigeo Hirose. Slip-based traction control for wheeled mobile robots. *Advanced Robotics*, 21(5–6):599–620, 2007. pages 14
- [5] Clive S Fraser. Automatic camera calibration in close range photogrammetry. *Photogrammetric Engineering & Remote Sensing*, 79(4):381–388, 2013. pages 31
- [6] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2004. pages 23
- [7] David Hsu, Nakul Gopalan, and Stefanie Tellex. Learning-based slip estimation for mobile robots using proprioceptive sensing. *IEEE Robotics and Automation Letters*, 3(4):3479–3486, 2018. pages 18
- [8] Karl Iagnemma and Steven Dubowsky. Mobile robot rough-terrain control (part i): Mechanical design, analysis, and experimental validation. *The International Journal of Robotics Research*, 23(1):25–42, 2004. pages 7, 14
- [9] Alonzo Kelly and Brian Nagy. Reactive nonholonomic trajectory generation via parametric optimal control. *The International Journal of Robotics Research*, 22(7–8):583–601, 2003. pages 33
- [10] Oussama Khatib. A unified approach for motion and force control of robot manipulators. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987. pages 8
- [11] Lauro Ojeda and Johann Borenstein. Methods for the reduction of odometry errors in overconstrained mobile robots. *Autonomous Robots*, 16(3):273–283, 2004. pages 7
- [12] Edwin Olson. Apriltag: A robust and flexible visual fiducial system. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 3400–3407, 2011. pages 23

-
- [13] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810, 2018. pages 15
 - [14] Joshua Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017. pages 15
 - [15] Corey C. Ward and Karl Iagnemma. Identification of wheel slip dynamics for mobile robots on rough terrain. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 5583–5588, 2008. pages 18